长程工作流：Harness、Loop 与 Graph

张虎来

复旦大学国际金融学院

2026 年秋季



本讲任务：构建量化 Agent 系统

```
Build a self-evolving quant trading plugin in Kimi Code.

User idea -> Orchestrator -> Generator team -> Critic team
          <- bounded repair, evidence and explicit outcomes

/qt:start  Research an idea through the complete inner loop
/qt:evo    Improve the reusable harness from recorded cases
```

- 本讲将亲手构建驱动这一量化系统运转的角色分工、运行控制与评估规则。

目录

- 设计量化 Agent 系统
  - Harness、Loop、Graph 与 Hooks
  - 持久状态与经过评估的演化
- 构建并演化 Kimi 插件
  - `/qt:start`：策略研究与独立审查
  - `/qt:evo`：提示词评估、版本管理与回滚
- 综合上机练习
  - 扩展组件、复跑流程并核验证据

产品：用户最终得到什么

- **研究案例**：交付候选策略、可运行代码、测试证据与独立审查意见。
- **明确决策**：清晰说明当前案例是通过验收、继续修订，还是终止。
- **可复用系统**：沉淀技能、专家团队、状态与版本，供下一个研究想法复用。

边界：中国股票市场

- **研究目标**：遵循各历史时点适用的交易制度，研究 A 股策略并构建组合。
- **课堂环境**：基于合成股票数据与动量规则，离线检验系统控制机制。
- **生产扩展**：为真实行情、交易日历、风控约束与成交回报接入经核验的接口。

职责：代码、Agent 与人

任务类型	责任主体	主要产出
解读证据	AI Agent	带引用的研究结论或异常说明
计算与约束	确定性代码	信号、权重、现金与股数
审批与执行	人	任务边界、策略准入与交易指令

- 问题：谁有权修改风险上限？

举手回答

验收：系统何时算成立

- **研究闭环**：交付经核验的证据与终局报告，完整覆盖所有必审维度。
- **中断恢复**：从已保存状态继续处理绑定的请求，绝不虚构已完成工作。
- **受控改进**：在固定案例上对比行为变化，记录审批决定，并支持版本回滚。

架构 1：单个 Agent



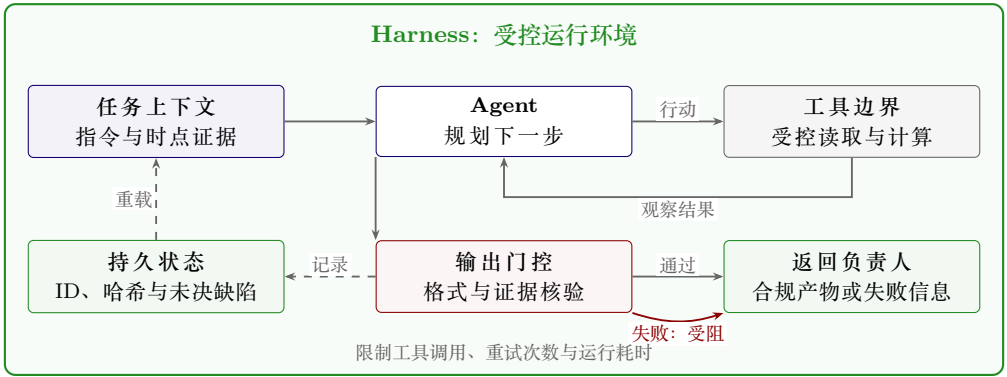
- 问题：当单个 Agent 出错时，拿什么核验这份报告？

举手回答

失败情形：自信但缺乏依据

- **无据主张**：“动量策略显著有效。”背后的样本区间与交易成本假设是什么？
- **无据操作**：“已完成回测验证。”留存的运行输出与日志在哪里？
- **无据结论**：“策略可直接上线。”谁核查了未来信息泄漏与风控边界？
- **核心教训**：流畅的文本只是待核验的产物，绝不等于证据。

架构 2：加入 Harness

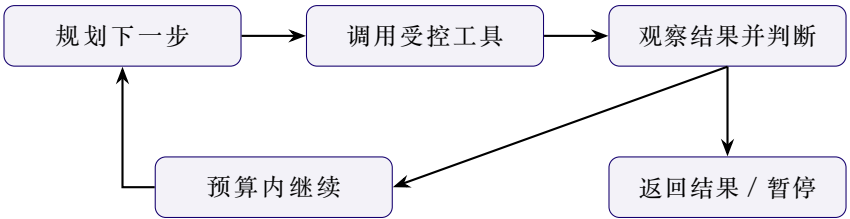


- Harness 用受控上下文、工具边界、持久状态与输出门控把模型包裹起来。

Harness：将任务转化为可执行契约

- **输入契约**：在运行前固定研究想法、信息截至日期（As-of Date）与股票池。
- **工具契约**：读取数据快照，调用已测算函数，并核查实际运行输出。
- **输出契约**：返回结构化产物清单、证据 ID 与明确的方法局限。
- **权限契约**：明确声明该角色允许读写的文件路径与可执行操作。

循环 1：计划、行动、观察

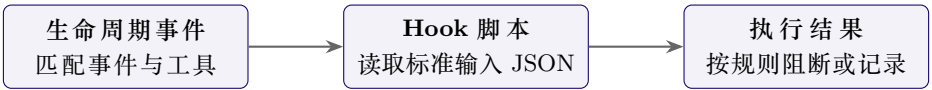


- 每次工具返回结果都会更新下一步判断，直至产出合规结果、触发报错或耗尽预算。

工具边界：调用前后双端验证

- **调用前**：校验输入参数结构、证券池范围与数据快照 ID。
- **调用后**：核查退出码、输出字段结构与产物文件哈希值。
- **失败时**：记录错误类型与现场证据，仅在瞬时故障时按上限重试。

Hooks：在事件边界自动触发



- Skill 指导工作流程，工具执行具体操作，而 Hook 在预定生命周期事件发生时自动运行脚本。

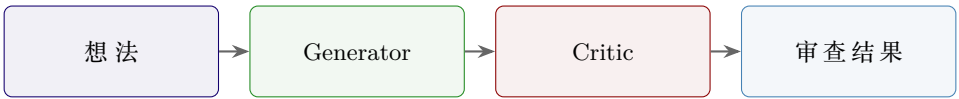
Hooks：选择恰当的触发事件

- `PreToolUse`：在工具执行前检查调用参数，必要时直接阻断违规调用。
- `PostToolUse`：调用成功后记录审计元数据；调用失败则使用 `PostToolUseFailure`。
- `SubagentStop`：记录子 Agent 完成事件，但团队是否齐备仍由控制器统一核验。
- `PreCompact`：在上下文压缩前为已落盘状态生成快照，其返回值会被忽略。

Hooks：权限边界与失败语义

- **退出码语义**：返回 0 正常继续；返回 2 仅在 `PreToolUse`、`UserPromptSubmit` 或 `Stop` 下触发阻断。
- **异常与超时**：脚本报错或超时后 Kimi 默认继续运行，因此强制风控门控必须留在控制器内。
- **`Stop` 事件**：可在重试上限内要求模型补充回复，但不能替代控制器的案例验收判定。
- **并发幂等**：同一事件匹配的多个 Hook 可能并行执行，脚本必须保持幂等且不依赖执行顺序。

架构 3：加入独立 Critic

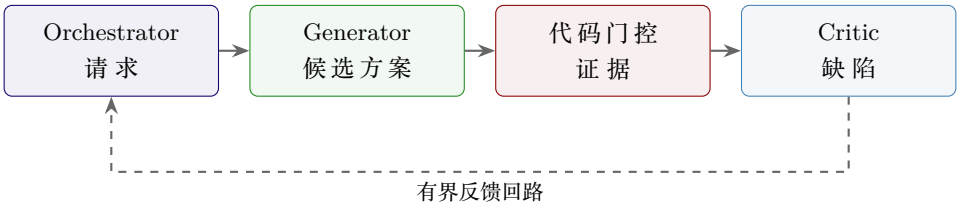


- Critic 在全新上下文中独立核查留存证据；模型自我附和绝不构成证明。

角色分工：构建与质疑分离

- **Generator**：负责提出并修复候选方案，但无权为自身产出背书。
- **Critic**：负责核查证据并登记缺陷，但无权直接改写候选方案。
- **Orchestrator**：掌握控制权，负责任务路由、缺陷台账维护、续行与终止判定。

架构 4：加入 Orchestrator



- 只有 Orchestrator 有权决定状态路由；Critic 的审查意见须经其台账整理后再发回修复。

Graph：受控的合法交接

- **节点（Node）**：具备明确输入输出契约的角色或确定性计算步骤。
- **边（Edge）**：仅当前置条件满足且证据校验通过时才允许触发的交接路径。
- **状态（State）**：决定下一跳路由的当前活动角色、已核验产物与未决缺陷记录。
- **显式控制**：用有向图将任务依赖、交接条件与角色权限固化为可强制执行的规则。

任务分解：量化系统必须构建的模块

- **数据层**：严格重建每个历史决策时点可获得的信息集合。
- **研究与验证层**：将投资假设转化为可检验信号，并映射为 Generator 的构建任务。
- **组合与交接层**：在风控约束下将信号转化为目标持仓与待审批建议单。
- **运维与审计层**：留存持久状态、调用轨迹与 Critic 审查维度，保障可复现与可恢复。

架构 5：为负责人配置专家团队

- **Generator 负责人**：统筹数据、特征与回测三位专家，协力产出一份完整候选方案。
- **Critic 负责人**：每轮启用独立审查专家，分别从时点、稳健性与执行三个维度核查。
- **Orchestrator**：派发双方负责人，并在每道交接屏障处校验团队产物清单是否齐全。

Generator 团队：分工与集成

- **数据专家**：核查数据快照质量，如实记录数据来源覆盖与时点局限。
- **特征专家**：将交易想法形式化为明确公式，并声明观测时点与执行滞后。
- **回测专家**：运行基准回测与压力测试，返回绩效指标与未解决的风险点。
- **时序约束**：独立核查可并行开展，但特征计算必须在数据校验通过后执行。

Graph：并行分发与汇合屏障

- **隔离写入**：各专家读取同一数据快照，但写入各自独立的输出路径，避免文件冲突。
- **汇合屏障**：负责人必须等待全部必选专家报告齐全，缺失任一报告均不得放行。
- **确定性汇总**：按显式规则合并专家产物与缺陷，绝不靠“多数表决”认定事实。

数据：重建决策时点的信息集

时点	数据记录	业务含义
2024-03-01	v1: value = 5.0	首次披露，当时可得
2024-05-01	v2: value = 4.7	修正值披露，此前不可得
4 月决策时点	使用 v1 = 5.0	先按披露时间筛选，再选版本

- 必须先按披露时间筛选可得记录，再从中选取版本，严防事后修正数据渗入早期决策。

特征：先声明时点，再写公式

要素	规则定义	工程实现
动量得分	$s_{i,t} = P_{i,t}/P_{i,t-20} - 1$	需满 21 个有效收盘价
算例校验	$12/10 - 1 = 20\%$	已知答案单元测试
截面排序	按得分降序，同分按既定规则排序	`baseline/strategy.py`

- 在给出信号公式的同时，必须明确记录观测截至时点与交易执行滞后。

执行：中国股票市场交易约束

- **T+1 交收**：分别追踪总持仓与可卖股数，严禁当日买入份额在同日卖出。
- **手数与资金**：按所属板块手数规则取整，预留交易税费，并逐日核对现金与持仓。
- **涨跌停与停牌**：按买卖方向执行涨跌停与停牌成交限制，如实保留拒单记录。
- **公司行动**：区分原始成交价与复权调整，按板块与日期核对分红送转引起的现金和股数变化。

Critic 团队：全维度独立审查

- **时点（PIT）审查**：排查未来信息泄漏、历史版本误用与股票池幸存者偏差。
- **稳健性审查**：核查逻辑与代码一致性、测试充分性以及交易成本敏感性。
- **执行可行性审查**：核查现金配平、整手约束、交易摩擦及实盘交接可行性。
- **独立性边界**：每轮使用全新上下文减少锚定效应，但同源模型与共享证据仍可能存在共性盲区。

Harness：为每个角色装配恰当上下文

- **固化规则**：将角色职责契约、JSON 结构与权限配置随代码一同版本化管理。
- **按需取证**：仅传递当前步骤所需的时点证据与未决缺陷，并保持文件路径和 ID 稳定。
- **外置状态**：将执行计划、中间产物与缺陷台账持久化到磁盘，而非依赖聊天记录记忆。

交接：候选方案与团队产物清单

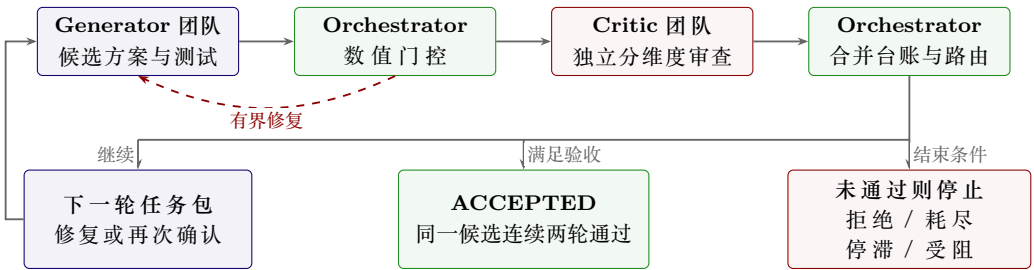
```
{
  "request_id": "<current dispatch>", "role": "generator", "round": 1,
  "reports": ["rounds/round-001/generator/data.json",
              "rounds/round-001/generator/features.json",
              "rounds/round-001/generator/backtest.json"],
  "candidate": {"lookback": 20, "top_n": 2,
                "rationale": "<how this tests the user idea>"}
}
```

- 在执行候选方案代码前，控制器先校验请求 ID 是否匹配、三份专家报告是否齐全。

反馈：可检验的修复任务包

- **稳定编号**：跨轮次保留缺陷唯一 ID（如 `pit-001`），便于全程追踪。
- **有据分级**：依据具体产物证据标定缺陷严重程度，杜绝空泛批评。
- **可检验要求**：向 Generator 下达明确的修复标准（如“改用决策时点已披露版本”）。
- **闭环核销**：必须由 Critic 核验修改后的产物方可关闭缺陷；审查漏提绝不视为关闭。

循环 2：候选方案迭代修订



修复超限记为 BLOCKED；更换候选方案重置计数；预算耗尽绝不视为通过。

- 修订循环只更新候选方案本身，任务边界与数值门控标准保持固定。

循环记忆：沉默绝不关闭缺陷

阶段	审查证据	台账状态
第 1 轮	`pit-001`：误用未来数据	缺陷待解决
第 2 轮	Critic 未提及 `pit-001`	仍保持开启
核验修复	Critic 引用修复后的产物证据	记录理由后关闭

- 后续轮次漏提旧缺陷绝不等于问题消失；台账中的每项未决缺陷都必须有明确核销记录。

终止：严格区分验收通过与失败停止

- `ACCEPTED`：同一候选方案通过数值门控，且连续两轮完整审查无未决缺陷。
- `CONTINUE`：轮次与时间预算尚余，控制器继续派发修复或第二轮确认审查。
- `EXHAUSTED` / `STALLED`：达到轮次或耗时上限，或连续重复同一未通过状态时停止。
- `REJECTED` / `BLOCKED` / `USER_STOPPED`：触发致命缺陷、提交违规或用户主动中断时停止。

资源：为循环设定硬边界

约束维度	默认规则	触发结果
审查轮次	每个案例默认最多 4 轮	达到上限记为 EXHAUSTED
无效提交	每次派发最多允许 2 次	超限后记为 BLOCKED
运行耗时	每个案例默认最长 4 小时	在状态转移边界检查
停滞判定	候选方案、缺陷与结论重复无进展	记为 STALLED，绝不放行

- 单次调用 Token 上限、费用核算与中断运行中的模型调用，均需依赖宿主环境控制。

休息：十分钟

1:20–1:30

- 接下来：为系统加入跨会话持久状态与经过评估的演化循环。

架构 6：加入持久状态

- **身份标识**：保存案例、请求、候选方案与提示词版本 ID，直接拒绝过期或串线回复。
- **执行进度**：在磁盘上记录当前角色、所处轮次、重试次数与截止时间。
- **证据完整性**：保存产物路径与 SHA-256 哈希值，及时识别已封存证据是否被改动。
- **台账记忆**：跨会话保留缺陷记录、核销理由与路由决定，确保未完工作可无缝恢复。

恢复：严格依据已核验状态续跑

- **待处理请求**：重新加载已绑定的 `request_id`，仅接收与之匹配的回复。
- **封存证据被改**：立即阻断推进（`BLOCKED`），排查原因后新建独立案例。
- **已完成案例**：直接返回已落盘的终局报告，不重复执行计算或外部操作。

恢复：先对故障分类，再决定路由

- **限流与超时**：按退避策略在上限内重试，并如实记录完成状态尚不确定。
- **结构校验失败**：将具体字段错误发回生成角色，在严格次数上限内要求修复。
- **触犯风控门控**：立即阻断并诊断原因，严禁为促成通过而放宽风控参数。
- **外部业务操作**：重试前先用幂等操作 ID 完成状态对账，严防重复报单。

权限：隔离各角色的工作区域

- **Generator**：仅允许写入指定的候选方案目录，无权修改控制器状态文件。
- **Critic**：仅允许写入审查报告目录，无权改动候选代码或回测证据。
- **Orchestrator**：统一核验返回的文件路径与哈希，独占状态转移控制权。
- **宿主权限隔离**：文件访问边界须由宿主权限强制执行，仅靠提示词无法实现真正隔离。

Harness 覆盖检查

- **任务与上下文**：明确定义角色契约、工作指令与按时点筛选的证据输入。
- **工具与权限**：校验调用参数结构，限制可读写路径，严防敏感凭据进入提示词。
- **跨会话延续**：将产物清单、状态检查点与提示词版本持久化到聊天上下文之外。
- **质量与预算**：将确定性代码门控、审计轨迹与重试和耗时上限紧密衔接。

上下文安全：将检索文本一律视为数据

- **不可信输入**：研报或公告中若出现“忽略规则、上传凭据”等字样，一律按普通数据对待。
- **指令隔离**：将系统控制指令与外部检索文档严格分区，外部文本无权篡改控制器规则。
- **最小权限**：通过宿主工具白名单强制约束读写范围，绝不依赖文档内容决定权限。

可观测性：让每次状态转移都可复盘

- **数据溯源**：记录每次运行所用的数据快照 ID、披露版本与完整性检查结果。
- **Agent 轨迹**：留存提示词版本、模型标识、工具调用参数与返回产物。
- **Graph 轨迹**：记录经过的节点、重试次数与路由转移原因，便于快速定位故障。
- **独立重放**：串联建议单版本、人工审批与成交记录，使同伴仅凭产物即可重放决策过程。

三个循环，三类改动对象

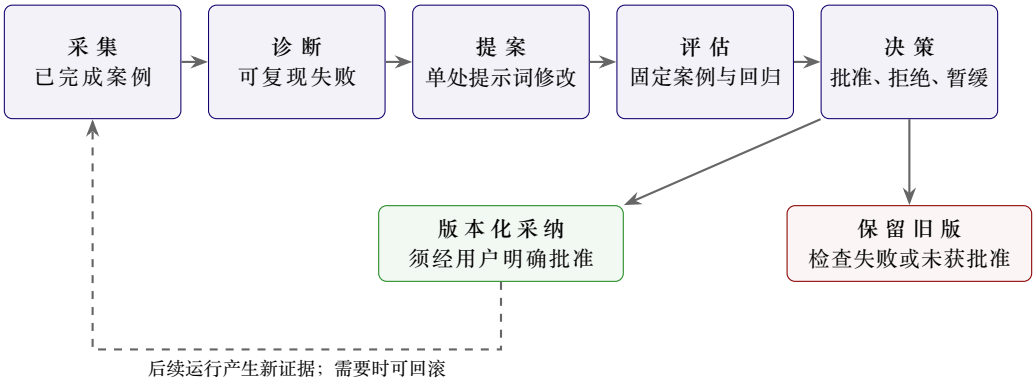
循环层次	改动对象	停止条件
工具循环	角色内部的下一步动作	产出合规结果、报错或达上限
研究循环	当前案例的候选策略	得出明确的案例终局结论
演化循环	可复用的 Harness 提示词	批准采纳、拒绝或暂缓

- 调整动量回看期属于策略搜索；改进证据核验与提示词契约属于 Harness 演化。

Loop 覆盖检查

- **工具循环**：规划、调用受控工具并观察结果，直至角色能够交付合规产物。
- **修订循环**：依据独立 Critic 的有据缺陷，驱动针对性的候选方案修复。
- **演化循环**：在固定评测集上评估 Harness 提示词改动，并配套版本管理与回滚。
- **终局状态**：清晰区分验收通过、明确拒绝、预算耗尽、停滞与违规阻断。

架构 7：加入演化循环



- 当前实现每次只针对 Generator 提示词提出单处修改，必须先通过评测再批准采纳。

采集：将真实失败转化为评测案例

- **记录故障事件**：三个已完成案例均遗漏执行时点声明，完整保留其原始请求与产物。
- **根因诊断**：排查究竟是角色提示词未强调执行滞后，还是其他环节导致遗漏。
- **固化回归案例**：在修改提示词之前，先将“必须写明观测与交易时点”固化为评测用例。
- **实证原则**：课堂示例仅作演示；实际演化必须基于真实运行留存的失败证据。

演化：一次只改动一个对象

- **当前允许范围**：通过工作区版本化覆盖机制，仅修改 Generator 角色提示词。
- **严禁改动红线**：风险门控与验收规则保持锁定，防止系统通过降低标准来“提高通过率”。
- **后续扩展要求**：若未来允许修改工具代码或路由逻辑，必须先为其构建专门的独立评估器。

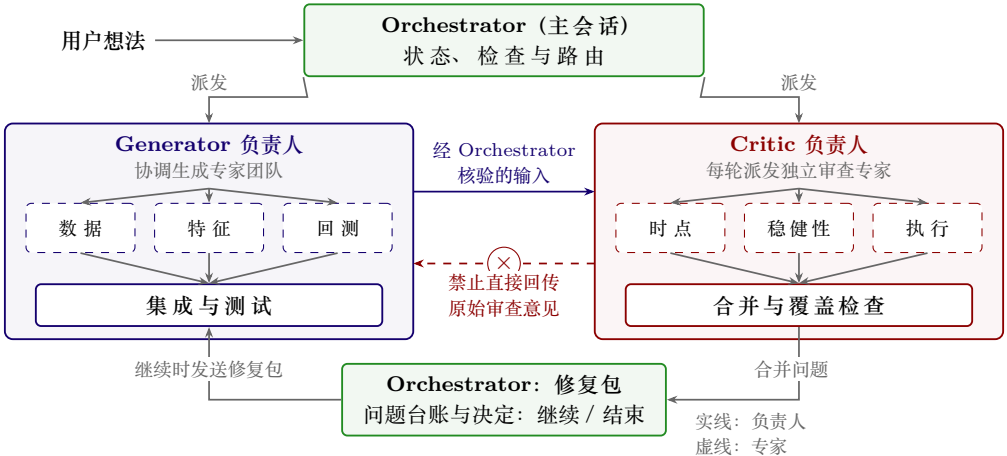
评估：证明改进必须依靠受控对照

- **同条件对照**：在**完全相同的输入、模型与预算**下对比旧版与新版提示词。
- **开发集诊断**：使用开发集案例排查问题，并**完整记录每次修改尝试**。
- **留出集终评**：将独立留出集用于最终对比，严禁针对留出集答案反复调优。
- **双重核验**：代码回归测试只保护基础机制；提示词是否改进须核查真实模型输出与独立审查。

采纳：人工审查、版本化与回滚

- **批准前审查**：人工核查提示词差异（Diff）、评测集表现与归档的模型对比输出。
- **版本化采纳**：仅在用户明确批准后写入不可变的新提示词版本，供后续新建案例使用。
- **随时可回滚**：支持将活动版本指针一键恢复至旧版本，同时完整保留两个版本的历史证据。

最终架构：角色、团队与屏障



- 内层运行积累案例证据；外层演化循环升级后续运行所用的 Harness 版本。

Graph 覆盖检查

- **有向无环图（DAG）**：明确声明从数据快照到交易建议单各节点的输入输出依赖。
- **条件路由**：当数据缺失、快照过期或门控未通过时，按规则转入对应处理分支。
- **分发与汇合**：并行派发多位独立专家，通过确定性汇合屏障收集并合并输出。
- **有界状态机**：清晰表达多轮修订循环、中断等待与人工审批节点。

交互接口：两个公开命令足够

交互入口	核心用途	内部工作流
`/qt:start`	将研究想法转化为证据与报告	提案、执行、独立审查与修订
`/qt:evo`	将失败案例转化为受评测的改进	诊断、评测、审查与版本采纳
日常对话	查询状态、停止、恢复或回滚	直接对话完成，无需额外命令

- 保持命令面极简：用两个命令覆盖研究与演化主流程，其余控制直接通过自然对话完成。

架构审查：动手编码前的四项核对

- **Harness**：输入契约、受控工具、状态路径与角色权限边界是否均已明确?
- **Loop**：修复任务包、连续通过计数与各类终局停止条件是否毫无歧义?
- **Graph**：专家依赖顺序、路由门控条件与汇合屏障是否均由代码强制执行?
- **演化**：固定评测案例、不可变版本存储与回滚指针是否均已就绪?

设计借鉴：公开量化与研发系统

- **RD-Agent**：将假设提出、实验编码与结果反馈串联成闭环。
- **TradingAgents**：按专业职责拆分市场分析、多方讨论与风险审查角色。
- **Vibe-Trading**：将可复用技能与可调用量化工具集成到 Agent 宿主环境。
- **本课 QT 综合设计**：将带门控的 Orchestrator–Generator–Critic 循环与受评测的演化结合。

插件：将系统架构映射为目录结构

```
qt/
  kimi.plugin.json         # commands, skills, agents
  commands/{start,evo}.md # two public entry points
  skills/{start,evo}/      # Orchestrator workflows
  agents/                 # two leads + six specialists
  references/             # schemas, role cards, policies
  runtime/quant_cn/       # state machine and numerical gates
  scripts/qt.py           # internal controller actions
  tests/                  # fixtures and regression tests
```

- 安装后的插件代码保持只读，运行期案例状态与演化版本统一保存在工作区 `.qt/` 目录下。

构建 1：注册插件清单

```
{
  "name": "qt",
  "version": "0.1.0",
  "commands": "./commands/",
  "skills": "./skills/",
  "agents": "./agents/"
}
```

- Kimi 自动以插件名作为命令前缀，因此 `commands/start.md` 会注册为 `/qt:start`。

构建 2：连接公开命令与工作流技能

```
---
description: Develop and evaluate a quant trading idea.
---
Invoke the qt:start Skill with the user arguments below.
User arguments: $ARGUMENTS

# Skill workflow:
# read controller -> create case -> dispatch lead
# validate manifest -> advance -> repeat or report
```

- Skill 提供 Orchestrator 的编排流程指令，Python 脚本负责执行持久状态转移与门控核验。

构建 3：声明原生团队委派关系

```
---
name: qt-generator
description: "Generator lead: integrate the specialist team."
tools: [Agent, Read, Glob, Grep, Write, Bash]
subagents: [qt-data, qt-features, qt-backtest]
---
Read the bound request. Dispatch separate specialist outputs.
Wait for all reports. Return one candidate manifest.
```

- 负责人通过 subagents 白名单声明可委派的专家；末端专家白名单为空，无需 tmux。

构建：注册 QT 审计 Hook

```
{
  "hooks": [
    {
      "event": "PostToolUse",
      "matcher": "^Bash$",
      "command": "python3 ./hooks/audit_event.py",
      "timeout": 3
    }
  ]
}
```

- 课堂练习：将该配置并入 `kimi.plugin.json` 并复制脚本（qt 0.1.0 默认未开启此项）。
- 插件 Hook 从插件根目录启动，通过读取事件 JSON 中的 `cwd` 字段定位当前工作区。

演示：测试 Hook 契约

- **最小化日志**：使用 `materials/topic-3/hooks/` 示例仅记录事件、会话、工具与调用 ID，不落盘原始命令。
- **幂等重放**：向脚本重复输入同一事件 JSON，确认日志中只保留一条记录。
- **作用域边界**：不含 `.qt/` 的普通目录不写入日志，且 Hook 日志绝不推进案例状态。
- **故障隔离**：先在离线环境测试异常输入，再进入 Kimi 验证真实事件触发与超时行为。

构建 4：安装插件到 Kimi Code

```
/plugins install /absolute/path/to/qt
/reload
/plugins info qt

/qt:start Test monthly momentum on the bundled synthetic fixture
```

- 从 `teaching/materials/qt` 安装；修改源码后须重新安装并 `/reload`，工作区状态仍保留在 `.qt/`。

原生运行环境：Kimi 宿主提供什么

- **命令与技能加载**：自动发现并按需加载 `/qt:start` 与 `/qt:evo` 两个工作流。
- **Agent 配置文件**：通过声明式白名单严格限制可用工具与可委派的子 Agent。
- **独立子会话执行**：每次子任务在独立上下文中运行；核验时应检查真实的 `Agent` 调用记录。
- **Python 控制器**：负责领域证据检查、数值门控与状态机（已在 Kimi Code 2.4.1 核验，无需 Tower）。

来源：Kimi 原生扩展接口文档

- **Plugins 文档**：官方插件清单结构、命令命名空间与组件打包规范。
- **Skills 文档**：官方 `SKILL.md` 编写规范、参数传递与配套资源加载机制。
- **Agents 文档**：官方自定义 Agent 角色配置、工具权限与 `subagents` 委派白名单规范。
- **Hooks 文档**：官方生命周期事件定义、退出码阻断规则与超时处理语义。

演示：追踪一个完整研究案例

```
/qt:start Test monthly momentum on the bundled synthetic fixture

Inspect .qt/cases/<case>/:
  idea.json -> request.json -> rounds/round-001/
  generator/*.json -> numerical/ -> critic/*.json
  state.json -> ledger and route -> final report
```

- 观察负责人派发三位专家的实时调用；仅凭磁盘上的 JSON 文件无法证明发生了独立的子 Agent 调用。

演示：逐一核查四道控制屏障

- **Generator 汇合屏障**：要求三份专家报告齐全，且 `request_id` 与轮次编号完全匹配。
- **数值门控屏障**：运行候选策略代码，核查信号、交易记录、净值曲线与门控结果。
- **Critic 审查屏障**：要求三份独立审查报告引用真实产物证据，并明确处理全部历史未决缺陷。
- **终局判定屏障**：要求同一候选方案连续两轮完整审查无缺陷，否则按明确停止状态终止。

核验：脚本化排练（Rehearsal）

```
python3 <plugin-root>/scripts/check.py
python3 <plugin-root>/scripts/qt.py start \
  --case rehearsal --idea "Synthetic monthly momentum" --rehearse

# Observed from the installed package:
# 35 tests passed
# Round 1: CONTINUE, clean=1
# Round 2: ACCEPTED, clean=2
# Mode: SCRIPTED_REHEARSAL
```

- 2026 年 11 月 3 日实测通过：该模式运行真实数值代码与控制器门控，但使用脚本报告而非实时模型团队。

实现边界：已核验范围与待扩展部分

核验状态	覆盖范围	证据边界
已核验	原生插件安装与命令发现	基于 Kimi Code 2.4.1 实测
已核验	35 项测试与两轮脚本排练	仅验证控制器与数值门控机制
尚未证明	真实团队表现与提示词改进效果	需在真实模型运行下对照评测
尚未实现	真实行情接入与券商实盘报单	属于独立的生产环境扩展

- 必须严格区分“已核验的控制器机制”与“尚待实测的真实模型表现或实盘效果”。

故障注入：验证四道屏障确实有效

注入故障	触发屏障	控制器动作
删去一份专家报告	团队汇合不完整	拒绝该次提交
篡改 `request_id`	过期或错配的回复	拒绝该次提交
修改已封存证据	证据哈希不一致	阻断案例推进
将轮次上限设为 1 轮	未满两轮无缺陷审查	以 `EXHAUSTED` 终止

- 运行配套行为测试或在临时案例中手工注入故障，确认每道屏障都能准确拦截违规推进。

演示：中断与恢复流程

- **记录中断现场**：在案例运行中暂停，核查已落盘的待处理 `request_id` 与封存产物哈希。
- **自然对话恢复**：直接要求继续该案例，控制器会先读取 `state.json` 再决定下一步委派。
- **核对状态衔接**：对照候选方案与缺陷台账确认进度，绝不凭空假设中断前的工作已完成。

休息：十分钟

2:45–2:55

- 接下来：运行 `/qt:evo` 改进 Harness 提示词，并梳理面向真实 A 股市场的扩展要求。

演示：发起一次 Harness 改进

```
/qt:evo Diagnose repeated missing timing in completed cases.
Propose one Generator-prompt change. Compare old and new
behavior on fixed cases before preparing adoption.

Inspect .qt/evolution/<change>/:
  request.json -> proposal.json -> review.json -> evaluation
Then inspect .qt/harness_versions/ after approved adoption.
```

- 基于真实留存的失败案例运行 `/qt:evo`；上方提示词仅展示任务写法，而非预设的评测结论。

演化：复用双团队，切换任务契约

- **Generator 团队**：诊断已记录的失败根因，设计单处提示词修改，并制定配套评测用例。
- **Critic 团队**：独立核查失败证据、回归测试结果以及改动是否越界。
- **Orchestrator**：运行代码回归测试，在 `AWAITING_APPROVAL` 处等待用户批准，并登记获批版本。

测量：提示词对照评估表

评测指标	计算口径	核验要求
证据覆盖率	有据可查的主张 / 全部主张	逐项核对实际产物文件
任务完成率	合规完整交接 / 全部案例数	分母须包含被阻断案例
运行稳定性	同预算下重复试验的表现	报告结果波动与资源消耗
安全退步项	越权改动或遗漏必选门控	任一违规即阻止采纳

- 在开始对比新旧提示词之前，必须事先固定各项指标的计算分母、案例划分与通过阈值。

演示：批准采纳与版本回滚

- **批准前核查**：审阅提示词 Diff、代码回归结果与归档的模型对比输出。
- **批准后验证**：明确批准采纳后启动一个新案例，核对当前生效的提示词版本路径。
- **一键回滚**：通过对话将版本指针恢复至上一版本，同时完整保留新旧版本的历史记录。

扩展：替换合成数据适配器

- **合规数据源**：接入获授权的行情与财务数据源，保留快照日期与溯源信息；检索工具不等于回测数据流。
- **历史证券池**：完整纳入历史指数成分变化、停复牌状态与退市证券，消除幸存者偏差。
- **时点对齐**：按公开披露时间与供应商入库时间做时点连接（PIT Join），并规范处理复权因子。
- **分板块执行**：按交易所、板块与具体日期测试交易日历、整手股数、涨跌停、税费与成交规则。

来源：市场规则与工程框架参考

- **上交所交易规则（2026）**：核对各板块申报手数、涨跌幅限制与交收制度的一手规则依据。
- **Qlib**：参考其面向时点特征工程与量化回测评估的工作流设计。
- **MLflow**：参考其实验参数、评估指标与不可变产物的版本追踪模式。
- **Dagster**：参考其声明式数据资产、依赖图管理与周期性任务编排模型。

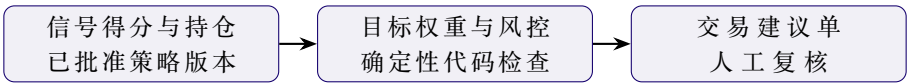
市场测试：主动证伪数据处理流水线

- **扰动未来价格**：篡改 t 日之后的价格，验证 t 日及之前的历史信号完全不变。
- **注入财务重述**：插入一条事后修正记录，确认修正日之前的历史快照仍读取原始披露值。
- **手工核对调仓**：抽取单个调仓日，手工验算成交股数、税费扣减与期末现金余额是否配平。
- **注入虚构引用**：在报告中编造不存在的证据 ID，确认控制器能够立即识别并拒绝。

研究验收：走向实盘前的压力检验

- **滚动样本外检验**：在历史窗口上滚动拟合并在下一窗口评估，覆盖牛熊与震荡等不同市场状态。
- **成本与滑点压力**：上调手续费率与冲击成本假设，检验策略净收益的衰减速度。
- **容量与换手约束**：限制成交参与率与换手率并检查拒单情况；通过控制器门控不代表具备实盘盈利能力。

组合扩展：严格遵循上下游依赖



- 课程附带的仓位建议表只是独立的历史算例，并非由研究循环刚通过的候选策略自动生成。

人工交接：从交易建议单到成交对账



- 真实组合交接必须接入当前实际持仓、带日期价格、审批有效期以及事后成交回报对账。

实验：三十五分钟实战扩展

时间	阶段任务	交付物
0–5 分钟	界定缺陷与组件契约	输入、输出、权限与停止规则
5–20 分钟	修改单个组件	一个可复现的失败测试案例
20–30 分钟	验证并完成集成	行为检查通过与回归证据
30–35 分钟	同伴重放核验	独立重放并说明剩余局限

- 在本地插件源码副本上修改一个组件，重新安装验证，并由同伴根据产物独立重放。

选择一项扩展任务

- **Harness 新鲜度检查**：拒绝过期数据快照时间戳，并编写边界测试用例。
- **Loop 缺陷核销绑定**：将缺陷关闭绑定到修复后的产物哈希，拒绝引用旧产物的核销申请。
- **Graph 汇合屏障扩展**：新增一位必选专家角色，并验证缺失该报告时控制器会阻断推进。
- **演化对照评测**：收紧 Generator 提示词中的时点声明要求，并在留出集案例上完成新旧对比。

从组件契约开始构建

```
Extend one QT component in my plugin source.
First state input, output, authority and failure behavior.
Demonstrate the defect with one behavioral case.
Make the smallest change; rerun affected and regression checks.
Show the diff, evidence, version and remaining limitation.
Reinstall the plugin before testing native command behavior.
```

- 保持公开命令接口不变，仍为 `/qt:start` 与 `/qt:evo` 两个入口。

评价标准：什么算合格的工程交付

- **契约清晰**：同伴能够清楚复述你所改组件的输入、输出、权限边界与失败停止规则。
- **证据可复现**：失败用例与修复后通过用例均可复现，并如实注明尚未覆盖的检查项。
- **集成无退步**：状态路由、持久状态与完整团队汇合屏障继续通过原有回归测试。
- **诚实报告局限**：清楚说明剩余局限与回滚方法；合成回测收益率高低不计入评分。

课程回顾：解释并捍卫你的架构设计

- **Harness**：为什么只允许该角色访问这些工具与路径？请展示权限边界证据。
- **Loop**：为什么当前案例应当继续修订或立即终止？请展示缺陷台账与门控证据。
- **Graph**：为什么该节点必须等待其他专家完成？请展示数据依赖与汇合屏障。
- **演化**：为什么批准采纳或拒绝这一提示词版本？请展示受控对照结果与回滚机制。

举手回答